

Звіт з лабораторної роботи №8

Тема: Bloom Filter

Мета роботи: дослідити принципи роботи ймовірнісних структур даних на прикладі фільтра Блума (Bloom Filter). Реалізувати програмну модель фільтра мовою Python, дослідити математичні закономірності визначення оптимальної кількості хеш-функцій (k) для мінімізації ймовірності хибнопозитивних спрацьовувань (false positives).

Виконав: студент групи ММШІ-1 Красіков Антон

Теоретичні відомості

Фільтр Блума - це компактна ймовірнісна структура даних, що використовується для перевірки належності елемента до множини. На відміну від класичних дерев (AVL, 2-3 дерева), які вимагають значних обсягів пам'яті для зберігання самих елементів, фільтр Блума зберігає лише бітові "відбитки".

Основні принципи:

1. Фільтр ініціалізується як масив із m бітів, встановлених у нуль.
2. Під час додавання елемента x до множини S , він обробляється k незалежними хеш-функціями. Отримані k позицій у бітовому масиві встановлюються в одиницю.
3. Під час перевірки належності x *in* S обчислюються ті самі k хешів. Якщо хоча б один відповідний біт дорівнює нулю, елемент гарантовано не додавався до множини. Якщо всі k бітів дорівнюють одиниці, елемент ймовірно знаходиться у множині.

Оптимізація кількості хеш-функцій (k):

Щоб мінімізувати ймовірність хибнопозитивного спрацювання (ϵ), оптимальна кількість хеш-функцій обчислюється через знаходження екстремуму функції та становить:

$$\min k = \frac{m}{n} \ln(2)$$

Спосіб вирішення

Для програмної реалізації було розроблено клас BloomFilter, який інкапсулює логіку роботи з бітовим масивом та хешуванням.

Інженерне рішення (Double Hashing):

Замість імпортування або написання k різних алгоритмів криптографічного хешування (що є обчислювально неефективним), було застосовано техніку подвійного хешування. Використовуються дві базові функції (MD5 та SHA1), на основі яких генерується будь-яка необхідна кількість псевдонезалежних хешів за формулою:

$$h_i(x) = (H_1(x) + i * H_2(x))(mod\ m)$$

Цей підхід є стандартом при реалізації фільтрів Блума в реальних системах. Клас також автоматично вираховує оптимальне k на етапі ініціалізації на базі заданих параметрів m та n .

Деталі реалізації в коді

Реалізація виконана мовою Python з використанням стандартної бібліотеки hashlib.

Основний клас BloomFilter:

```

class BloomFilter:
    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def __init__(self, expected_items: int, bit_array_size: int):
        self.n = expected_items
        self.m = bit_array_size

        # Calculate optimal 'k' based on the formula:  $\min k = (m / n) * \ln(2)$ 
        # We use max(1, ...) because we need at least 1 hash function
        optimal_k = (self.m / self.n) * math.log(2)
        self.k = max(1, int(round(optimal_k)))

        # Initialize bit array with 0s
        self.bit_array = [0] * self.m
        self.logger = logging.getLogger("BloomFilter")

        self.logger.info(
            f"Initialized Filter -> Array size (m): {self.m}, "
            f"Expected items (n): {self.n}, Optimal hashes (k): {self.k}"
        )

```

```

    Windsurf: Refactor | Explain | ✕
    def _get_hashes(self, item: str) -> list[int]:
        """
        Uses Double Hashing technique to simulate 'k' hash functions
        efficiently without calculating 'k' cryptographic hashes.
        """

        encoded_item = item.encode('utf-8')

        # Get two base hashes (using standard libraries for demonstration)
        hash1 = int(hashlib.md5(encoded_item).hexdigest(), 16)
        hash2 = int(hashlib.sha1(encoded_item).hexdigest(), 16)

        # Generate k hashes: (hash1 + i * hash2) % m
        return [(hash1 + i * hash2) % self.m for i in range(self.k)]

```

```

    Windsurf: Refactor | Explain | ✕
    def add(self, item: str) -> None:
        """Adds an item to the Bloom filter."""
        hashes = self._get_hashes(item)
        for h in hashes:
            self.bit_array[h] = 1
        self.logger.debug(f"Item '{item}' added at positions: {hashes}")

```

```

    Windsurf: Refactor | Explain | ✕
    def check(self, item: str) -> bool:
        """
        Checks if an item is in the set.
        Returns False -> 'definitely not in S'
        Returns True -> 'probably in S'
        """

        hashes = self._get_hashes(item)
        for h in hashes:
            # If any corresponding bit is 0, the item was never added
            if self.bit_array[h] == 0:
                return False
        return True

```

Скрипт симуляції роботи (тестування запитів):

```
if __name__ == "__main__":
    # Example: We expect to store 10 items, and we allocate 100 bits of memory.
    bf = BloomFilter(expected_items=10, bit_array_size=100)

    # Add some data
    words_to_add = ["apple", "banana", "cherry"]
    for word in words_to_add:
        bf.add(word)

    print("\nTesting Queries...")

    # Test items we know are in the filter (should be True)
    for word in words_to_add:
        result = bf.check(word)
        print(f"Query '{word}': {result} (Expected: True -> probably in S)")

    # Test items we did not add (should be False, but True is a false positive)
    words_not_added = ["grape", "orange", "pineapple"]
    for word in words_not_added:
        result = bf.check(word)
        print(f"Query '{word}': {result} (Expected: False -> definitely not in S)")
```

Результати виконання

Під час виконання програми було ініціалізовано фільтр Блума з параметрами $m = 100$ та очікуваною кількістю елементів $n = 10$. Алгоритм автоматично вирахував мінімально необхідну кількість хеш-функцій: $k = 7$.

Вивід консолі (логи виконання):

```
INFO | Initialized Filter -> Array size (m): 100, Expected items (n):
10, Optimal hashes (k): 7
```

```
Testing Queries...
```

```
Query 'apple': True (Expected: True -> probably in S)
```

```
Query 'banana': True (Expected: True -> probably in S)
```

```
Query 'cherry': True (Expected: True -> probably in S)
```

```
Query 'grape': False (Expected: False -> definitely not in S)
Query 'orange': False (Expected: False -> definitely not in S)
Query 'pineapple': False (Expected: False -> definitely not in S)
```

Аналіз результатів:

Система успішно додала вказані елементи. Перевірка існуючих елементів коректно повернула True (збереження цілісності даних, відсутність хибнонегативних результатів). Перевірка відсутніх елементів повернула False, оскільки при заданих параметрах ($m=100$, $n=10$) заповненість бітового масиву була достатньо низькою, щоб уникнути колізій хешів для тестової вибірки.

Висновки

В ході виконання роботи була успішно реалізована програмна модель фільтра Блума. Математично та практично підтверджено, що для забезпечення максимальної ефективності структури даних необхідно динамічно розраховувати параметр k (кількість хеш-функцій) в залежності від відношення виділеної пам'яті (m) до кількості елементів (n).

Фільтр Блума демонструє $O(k)$ часову складність для операцій вставки та пошуку, що робить його незамінним інструментом у високонавантажених системах (наприклад, для уникнення непотрібних звернень до повільних дискових баз даних). Оптимізація через подвійне хешування дозволила зробити реалізацію обчислювально легкою та готовою до масштабування. Єдиним обмеженням базової версії фільтра є неможливість видалення елементів (через ризик пошкодження інших записів), що вимагає використання

модифікацій (наприклад, Counting Bloom Filter) у випадках, коли множина є динамічною.

Посилання на код (папка lab8):

<https://github.com/chesswondo/distributed-systems-labs>